

Personal Music Library & Playlist Manager

Backend Project Planning Assignment

Problem Statement

Who has the problem? People who listen to music across multiple platforms and devices, and want one place to organize their own library — favorite songs, custom playlists, and artists they follow — instead of relying solely on a streaming app's algorithm.

Why is it frustrating? Playlists get scattered across different apps (Spotify, YouTube Music, local downloads). It is hard to track personal listening habits, such as most played songs or favorite artists, without digging through app menus. Streaming apps often limit playlist organization — for example, no folders/categories for playlists and no way to add personal notes or ratings on songs.

How is it currently solved? People rely on the built-in playlist features of apps like Spotify or Apple Music.

Why is the current solution not good enough? Users do not own or control their data — switching apps means playlists do not transfer easily. There is little customization, such as tagging songs with personal moods or categories, or viewing simple personal stats like “songs played most this month.” There is also no way to combine music from different sources into a single personal catalog.

Target Users

- Individual music listeners who want a personal, portable library independent of any single streaming platform.
- Users who enjoy organizing music by mood, genre, or custom categories.
- Users interested in tracking their own listening habits and statistics.

Proposed Solution

A backend system where users manage their own personal music library, independent of any single streaming app. It lets users add songs, organize them into custom playlists, tag playlists by mood or category, track how often they play songs, and view personal listening statistics such as top songs and top artists.

What the system does: Stores a user's songs, artists, and playlists; tracks play counts; generates simple listening reports.

Who will use it: Individual music listeners who want a personal, portable library.

What makes it useful: Centralized control over personal music data, mood-based organization, and personal statistics not tied to any single streaming platform.

What data it stores: Users, Songs, Artists, Playlists, playlist-song associations, and play history.

Core Features (MVP)

- User registration and login
- Add, edit, and delete songs
- Create, update, and delete playlists

- Add or remove songs from a playlist
- Search songs by title, artist, or genre
- Tag playlists with a mood or category
- Track play count per song
- View reports: top played songs, top artists

Entities and Their Fields

User

Field	PostgreSQL Type
id	BIGSERIAL (PK)
username	VARCHAR(50) UNIQUE NOT NULL
email	VARCHAR(255) UNIQUE NOT NULL
passwordHash	VARCHAR(255) NOT NULL
createdAt	TIMESTAMP DEFAULT now()

Artist

Field	PostgreSQL Type
id	BIGSERIAL (PK)
name	VARCHAR(255) NOT NULL
genre	VARCHAR(100)
bio	TEXT

Song

Field	PostgreSQL Type
id	BIGSERIAL (PK)
title	VARCHAR(255) NOT NULL
artistId	BIGINT (FK → Artist.id)
genre	VARCHAR(100)
duration	INTEGER
releaseDate	DATE
playCount	INTEGER DEFAULT 0

Playlist

Field	PostgreSQL Type
id	BIGSERIAL (PK)

name	VARCHAR(255) NOT NULL
description	VARCHAR(500)
mood	VARCHAR(50)
userId	BIGINT (FK → User.id)
createdAt	TIMESTAMP DEFAULT now()

PlaylistSong (join entity)

Field	PostgreSQL Type
id	BIGSERIAL (PK)
playlistId	BIGINT (FK → Playlist.id)
songId	BIGINT (FK → Song.id)
addedAt	TIMESTAMP DEFAULT now()
position	INTEGER

PlayHistory

Field	PostgreSQL Type
id	BIGSERIAL (PK)
userId	BIGINT (FK → User.id)
songId	BIGINT (FK → Song.id)
playedAt	TIMESTAMP DEFAULT now()

Entity Relationships

- One User can create many Playlists.
- One Playlist can contain many Songs, and one Song can appear in many Playlists — a many-to-many relationship implemented through the PlaylistSong join entity.
- One Artist can have many Songs.
- One User has many PlayHistory records, and one Song has many PlayHistory records.
- (Stretch) One User can follow many Artists, and one Artist can be followed by many Users — a many-to-many relationship.

REST API Design

Users

POST /users — Register a new user

POST /auth/login — Log in

GET /users/{id} — Get user details

PUT /users/{id} — Update user details

Artists

POST /artists — Create an artist
GET /artists — List all artists
GET /artists/{id} — Get artist details

Songs

POST /songs — Add a new song
GET /songs — List all songs
GET /songs/{id} — Get song details
PUT /songs/{id} — Update a song
DELETE /songs/{id} — Delete a song
GET /songs/search?query= — Search songs by title/artist/genre
POST /songs/{id}/play — Increment play count

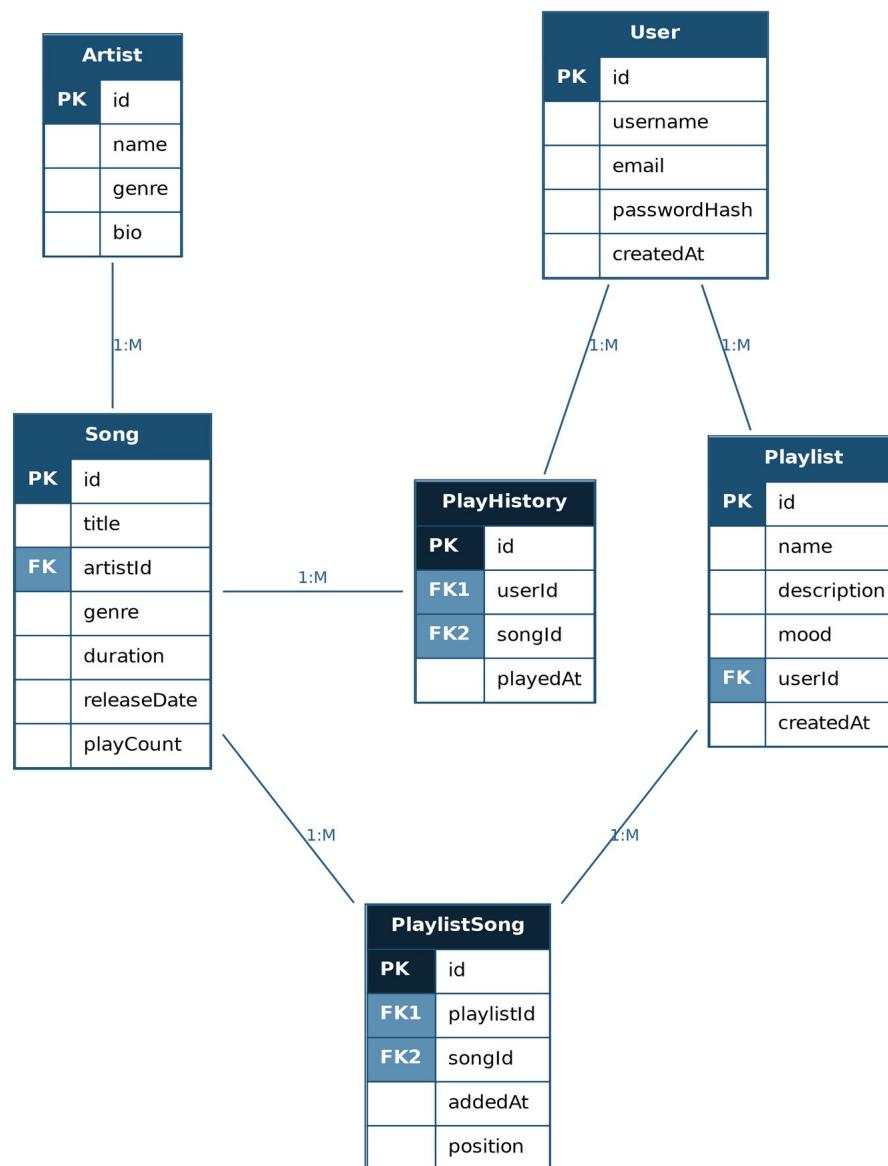
Playlists

POST /playlists — Create a playlist
GET /playlists — List all playlists
GET /playlists/{id} — Get playlist details
PUT /playlists/{id} — Update a playlist
DELETE /playlists/{id} — Delete a playlist
POST /playlists/{id}/songs — Add a song to a playlist
DELETE /playlists/{id}/songs/{songId} — Remove a song from a playlist

Reports

GET /reports/top-songs — Top played songs
GET /reports/top-artists — Top artists by play count

Database ERD



Relationship summary:

- Users (1) → (M) Playlists
- Playlists (1) → (M) PlaylistSongs ← (M) (1) Songs [many-to-many via join table]
- Artists (1) → (M) Songs
- Users (1) → (M) PlayHistory ← (M) (1) Songs

Future Improvements (Optional)

- Follow/unfollow artists (via an ArtistFollow join entity)
- JWT-based authentication and authorization
- Scheduled job to refresh a “Top Played” materialized view nightly
- Caching for frequently requested reports
- Rate limiting on the /songs/{id}/play endpoint

- Music recommendations based on play history
- Import songs via an external API (e.g., Spotify)

Appendix: Additional Design Notes (Bonus)

Not required by the assignment, but included to show deeper backend thinking beyond the MVP scope.

Authorization Rules

- A user may create, update, and delete only their own Playlists.
- A user may add or remove songs only on playlists they own.
- PlayHistory records are scoped to the authenticated user.
- Song and Artist catalog data is shared/global and readable by any authenticated user; edits are admin-only.
- All endpoints except POST /users and POST /auth/login require authentication (401 if missing, 403 if accessing another user's private resource).

Data Integrity & Performance Notes

- Unique composite index on PlaylistSong(playlistId, songId) to prevent duplicates.
- Index on PlayHistory(userId, playedAt) for time-windowed report queries.
- Index on Song(artistId) and a search index on Song(title) / Artist(name) for GET /songs/search.
- ON DELETE CASCADE on PlaylistSong and PlayHistory foreign keys.

Note on playCount

Song.playCount is a denormalized counter kept in sync with PlayHistory in the same transaction on POST /songs/{id}/play — used for fast lifetime-total lookups, while PlayHistory remains the source of truth for time-based queries.